

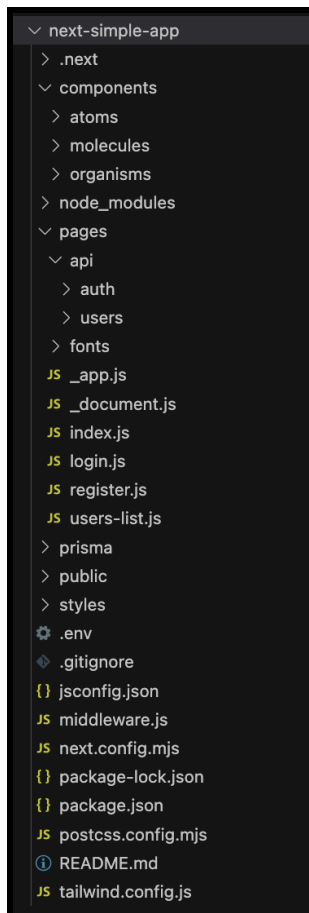
DOKUMENTASI PENGEMBANGAN WEBSITE SEDERHANA MENGUNAKAN FRAMEWORK NEXTJS DAN TAILWINDCSS DI BRI DevStudio

A. Deskripsi

Aplikasi ini adalah sebuah website yang dikembangkan menggunakan framework NextJS dan Tailwindcss serta DBMS PostgreSQL di Platform BRI DevStudio. Aplikasi ini bertujuan untuk memberikan pengalaman pengembangan software menggunakan platform BRI DevStudio. Fitur-fitur yang tersedia pada aplikasi ini meliputi *user authentication* seperti *login* dan *logout* menggunakan JWT Token dan *users list*.

B. Struktur Proyek

Berikut ini struktur proyek pada website *Simple Next App* di BRI DevStudio:



Penjelasan Folder / File dalam struktur proyek :

1. **Folder .next** : folder build yang dihasilkan secara otomatis oleh Next.js ketika menjalankan proyek (`npm run build` atau `npm run dev`). Folder ini berisi berkas yang digunakan untuk menjalankan aplikasi Next.js, seperti cache, compiled JavaScript, dan aset build. Folder ini adalah hasil build internal yang dihasilkan oleh Next.js.

2. Folder **components**

Beberapa folder yang ada di components diantaranya yaitu :

- **atoms**: bagian terkecil dalam pola **Atomic Design**. Folder **atoms** biasanya berisi komponen UI dasar yang tidak dapat dipecah lebih lanjut. Contohnya adalah `Button.js`, yang mungkin hanya komponen tombol sederhana dengan styling.
- **molecules**: bagian yang lebih besar dalam pola **Atomic Design**. Komponen di dalam folder **molecules** merupakan kombinasi dari beberapa "atoms". Contohnya, `LoginForm.js` mungkin adalah form login yang terdiri dari input email, password, dan tombol submit (komponen-komponen ini mungkin berada di folder **atoms**).
- **organisms**: Ini adalah komponen yang lebih kompleks dan biasanya terdiri dari beberapa **molecules** atau bahkan **atoms**. Misalnya, `AuthForm.js` dapat berisi keseluruhan form login yang mungkin mencakup beberapa bagian lainnya.
- **Kesimpulan**: Penggunaan folder ini menunjukkan bahwa project ini dibuat menggunakan pendekatan **Atomic Design** dalam pembuatan komponen UI. Pendekatan ini memungkinkan kode menjadi lebih modular, dan dapat digunakan kembali (reusable), dan mudah dikelola.

3. Folder **pages**

- **api**: Folder **api** di dalam **pages** adalah tempat untuk mendefinisikan **API Routes** di Next.js. Semua file di dalam folder ini secara otomatis menjadi endpoint API.
 - a. **auth**: Folder ini berisi logika untuk autentikasi pengguna, seperti:

- **login.js**: Endpoint API untuk menangani login pengguna.
- **register.js**: Endpoint API untuk menangani pendaftaran pengguna.
- **users**: Di sini mungkin berisi API untuk mengakses atau memanipulasi data pengguna. Misalnya, endpoint yang mengembalikan daftar pengguna.
- **index.js**: Ini adalah halaman utama yang diakses ketika pengguna mengunjungi URL `/`. Dalam aplikasi ini, **index.js** berisi logika redirect berdasarkan status autentikasi pengguna.
- **login.js**: Halaman login tempat pengguna bisa memasukkan email dan password untuk masuk.
- **register.js**: Halaman pendaftaran tempat pengguna bisa mendaftarkan akun baru.
- **users-list.js**: Halaman ini mungkin digunakan untuk menampilkan daftar pengguna yang sudah terdaftar dalam aplikasi.
- **Kesimpulan**: Next.js menggunakan pendekatan **file-based routing**, di mana setiap file di dalam folder **pages** secara otomatis menjadi rute di aplikasi. Dengan demikian, struktur folder ini menentukan bagaimana halaman dan API tersedia di aplikasi.

4. Folder **prisma**

- **migrations**: Folder ini berisi skrip migrasi yang dihasilkan oleh Prisma saat menjalankan `npx prisma migrate dev`. Skrip migrasi ini digunakan untuk mengubah atau membuat tabel di dalam database.
 - a. **20240918075539_init**: contoh folder migrasi yang dihasilkan dengan nama timestamp ketika migrasi pertama kali dijalankan. Di dalamnya ada berkas **migration.sql** yang berisi SQL untuk membuat tabel atau modifikasi lainnya pada database.
- **schema.prisma**: Ini adalah file utama Prisma yang mendefinisikan model database. Prisma menggunakan file ini untuk menghasilkan query builder dan tipe TypeScript yang sesuai dengan model database.
- **Kesimpulan**: project ini menggunakan **Prisma** sebagai **ORM (Object-Relational Mapping)** untuk menghubungkan aplikasi Next.js

dengan database. Ini memudahkan dalam menangani query database tanpa menulis SQL secara langsung.

5. Folder **public**

- **Favicon.ico:** icon aplikasi yang ditampilkan di tab browser ketika pengguna mengunjungi aplikasi.
- **Kesimpulan:** Folder **public** berisi file statis seperti gambar, ikon, atau file lainnya yang dapat diakses langsung di URL tanpa proses build. Misalnya, **favicon.ico** akan diakses di **/favicon.ico**.

6. Folder **styles**

- **globals.css:** tempat untuk menyimpan **global styles** yang digunakan di seluruh aplikasi. Dalam proyek Next.js, file ini biasanya diimpor di **_app.js** agar styling global diterapkan ke semua halaman.
- **Kesimpulan:** Next.js memungkinkan penggunaan CSS atau CSS-in-JS. File **globals.css** digunakan untuk mendefinisikan styling global yang berlaku untuk semua halaman.

7. File **.env**

- adalah file **environment variables**. digunakan untuk menyimpan variabel-variabel seperti **DATABASE_URL**, **JWT_SECRET**, atau **NEXT_PUBLIC_URL**, yang digunakan oleh aplikasi.
- **Kesimpulan:** File ini berguna untuk menyimpan informasi sensitif atau konfigurasi yang bervariasi antara environment (development, staging, production) tanpa menyimpan variabel tersebut langsung dalam kode.

8. File **_app.js**

- File ini digunakan untuk **custom App component** di Next.js. Semua halaman di dalam folder **pages/** akan menggunakan komponen ini sebagai "wrapper". Biasanya digunakan untuk mendefinisikan state global, provider (misalnya **React Context** atau **Redux**), atau styling global.
- **Kesimpulan:** File **_app.js** memastikan bahwa setiap halaman berbagi setup atau styling yang sama, misalnya styling TailwindCSS atau provider dari **react-toastify** untuk menampilkan toast.

9. File **_document.js**

- File ini adalah tempat untuk meng-custom rendering dokumen HTML. Digunakan untuk menambah atribut tambahan di elemen `html` atau `body`, atau menambahkan meta tag global seperti `lang`, `charset`, dll.
- **Kesimpulan:** File ini jarang digunakan kecuali perlu memodifikasi elemen dasar HTML secara global di seluruh halaman.

10. File `middleware.js`

- File ini digunakan untuk menangani middleware di Next.js. Middleware ini mungkin digunakan untuk mengarahkan pengguna ke halaman login jika mereka belum login, atau mencegah akses ke halaman tertentu.
- **Kesimpulan:** Middleware di Next.js dapat digunakan untuk autentikasi, logging, atau manipulasi request sebelum diteruskan ke halaman atau API.

11. File Konfigurasi

- **`next.config.mjs`:** File ini berisi pengaturan dan konfigurasi untuk Next.js. Ini memungkinkan untuk mengubah perilaku default Next.js, seperti custom routing, build, atau pengaturan environment.
- **`tailwind.config.js`:** File konfigurasi untuk **TailwindCSS**. TailwindCSS adalah framework CSS utility-first yang memungkinkan membuat UI yang responsif dan cepat tanpa menulis banyak CSS secara manual. File ini digunakan untuk menyesuaikan tema Tailwind seperti warna, ukuran, dan breakpoint.
- **`postcss.config.mjs`:** Konfigurasi untuk **PostCSS**, yang sering digunakan bersama TailwindCSS untuk menangani CSS processing.

B.1 Framework Yang Digunakan

Framework yang digunakan untuk mengembangkan website ini yaitu :

1. Next.js (Page Router)

Next.js adalah **framework React** yang digunakan untuk membangun aplikasi web yang dirender di server (server-side rendering) atau dirender di klien (client-side rendering). Next.js memiliki banyak fitur canggih seperti **routing berbasis halaman, API routes, server-side rendering (SSR), static site generation (SSG), dan incremental static regeneration (ISR)**.

Next JS yang digunakan untuk mengembangkan aplikasi menggunakan konsep Page Router, maksud dari Next JS Page router adalah rute ditentukan oleh struktur file di dalam folder `pages/`. Setiap file JavaScript di dalam folder `pages/` secara otomatis akan menjadi rute di aplikasi web. Misalnya:

- `pages/index.js` akan menjadi rute untuk `/`
- `pages/login.js` akan menjadi rute untuk `/login`

Tidak perlu konfigurasi tambahan untuk routing karena Next.js secara otomatis menangani semua rute berdasarkan struktur folder.

a. Mengapa Menggunakan Next.js?

- **File-based routing:** Routing yang sangat mudah diatur dan dikelola.
- **Server-side rendering (SSR)** dan **static site generation (SSG)** memberikan fleksibilitas untuk memilih bagaimana halaman di-render.
- **API routes:** bisa membuat API backend langsung di dalam aplikasi Next.js tanpa memerlukan server terpisah.
- **Optimasi performa:** Next.js melakukan optimasi seperti image optimization, code-splitting, dan lazy loading secara otomatis.

2. Atomic Design (Prinsip Desain)

Atomic Design adalah metodologi yang dikembangkan oleh **Brad Frost** untuk merancang dan membangun antarmuka pengguna (UI) yang modular dan scalable. Pendekatan ini membagi komponen UI menjadi bagian yang lebih kecil dan reusable, yang memungkinkan desain menjadi lebih terstruktur dan konsisten.

a. Struktur Atomic Design:

- **Atoms:** Komponen terkecil yang tidak dapat dipecah lebih lanjut. Contohnya, sebuah tombol (**Button.js**), input field (**Input.js**), atau label (**Label.js**).
- **Molecules:** Kombinasi dari beberapa **Atoms** yang membentuk komponen yang lebih kompleks. Misalnya, form login yang berisi input email, input password, dan tombol submit (**LoginForm.js**).
- **Organisms:** Kombinasi dari beberapa **Molecules** dan **Atoms**. Contohnya adalah header yang terdiri dari navigasi dan logo, atau form login lengkap (**AuthForm.js**).
- **Templates:** Template adalah kombinasi dari **Organisms** yang membentuk layout halaman atau bagian dari halaman.
- **Pages:** Halaman lengkap yang dihasilkan oleh **Templates**. Di sinilah konten spesifik diisi untuk halaman tersebut.

b. Mengapa Menggunakan Atomic Design?

- **Reusability:** Komponen yang dirancang berdasarkan Atomic Design dapat digunakan kembali di berbagai bagian aplikasi, sehingga mengurangi duplikasi kode.
- **Scalability:** Desain dapat berkembang secara lebih terstruktur saat proyek semakin besar.
- **Consistency:** Desain yang konsisten di seluruh aplikasi karena komponen UI bersifat modular.

3. TailwindCSS (CSS Framework)

TailwindCSS adalah **CSS utility-first framework** yang memungkinkan membangun antarmuka pengguna secara cepat tanpa menulis CSS yang berlebihan. Dengan TailwindCSS, tidak perlu membuat banyak file CSS khusus untuk setiap komponen; cukup menggunakan kelas-kelas yang sudah disediakan untuk styling.

a. Cara Kerja TailwindCSS:

TailwindCSS memberikan utility classes yang memungkinkan mengatur styling langsung di dalam HTML atau JSX dengan menambahkan kelas-kelas seperti `bg-blue-500`, `text-center`, `p-4`, `m-2`, dll. Setiap kelas ini melakukan satu hal spesifik, misalnya:

- **bg-blue-500**: Memberikan background warna biru.
- **text-center**: Menyelaraskan teks ke tengah.
- **p-4**: Menambahkan padding sebesar 1rem.

b. Fitur Utama:

Beberapa fitur utama diantaranya yaitu :

- **Utility-first**: Alih-alih membuat komponen dengan CSS khusus, menggunakan utility classes yang sudah disediakan oleh Tailwind.
- **Responsive design**: TailwindCSS memiliki dukungan bawaan untuk desain responsif dengan breakpoint seperti `sm`, `md`, `lg`, dll.
- **Purge CSS**: Tailwind dapat menghapus kelas-kelas yang tidak digunakan dalam proses build, sehingga ukuran CSS di production sangat kecil.

c. Mengapa Menggunakan TailwindCSS?

- **Produktivitas tinggi**: dapat membangun dan mengubah styling dengan cepat tanpa harus berpindah ke file CSS terpisah.
- **Kustomisasi penuh**: Tailwind menyediakan file konfigurasi (`tailwind.config.js`) di mana bisa menyesuaikan tema, warna, dan ukuran sesuai kebutuhan aplikasi.
- **Responsiveness bawaan**: bisa dengan mudah mengatur styling responsif dengan class utility tanpa menulis media query CSS manual.

- **Scalability:** Dengan Tailwind, scaling aplikasi menjadi lebih mudah karena utility classes memastikan konsistensi di seluruh proyek.

4. Prisma (Database ORM)

Prisma adalah **Object-Relational Mapping (ORM)** yang mempermudah interaksi antara aplikasi Node.js dan database relasional seperti PostgreSQL, MySQL, dan SQLite. Prisma memungkinkan untuk mengelola database menggunakan model yang didefinisikan di dalam file `schema.prisma` dan menyediakan query builder yang aman secara tipe di TypeScript.

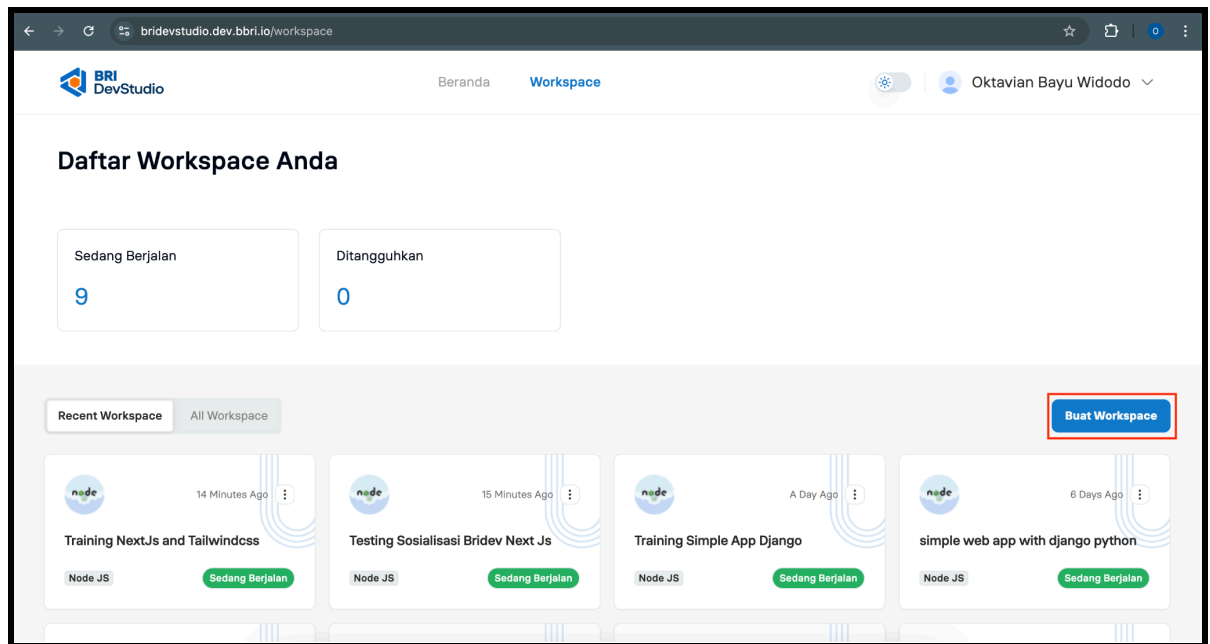
a. Komponen Utama Prisma:

- **Prisma Client:** Ini adalah query builder yang dihasilkan berdasarkan model yang didefinisikan dalam `schema.prisma`. Dengan Prisma Client, dapat melakukan query ke database seperti `find`, `create`, `update`, dan `delete` tanpa menulis SQL secara manual.
- **Prisma Migrate:** Alat ini memungkinkan mengelola migrasi database. Saat mengubah skema (misalnya, menambah kolom baru), Prisma akan menghasilkan skrip SQL yang sesuai untuk menerapkan perubahan tersebut ke database.
- **Prisma Studio:** Antarmuka GUI untuk melihat dan mengedit data di database dengan mudah.

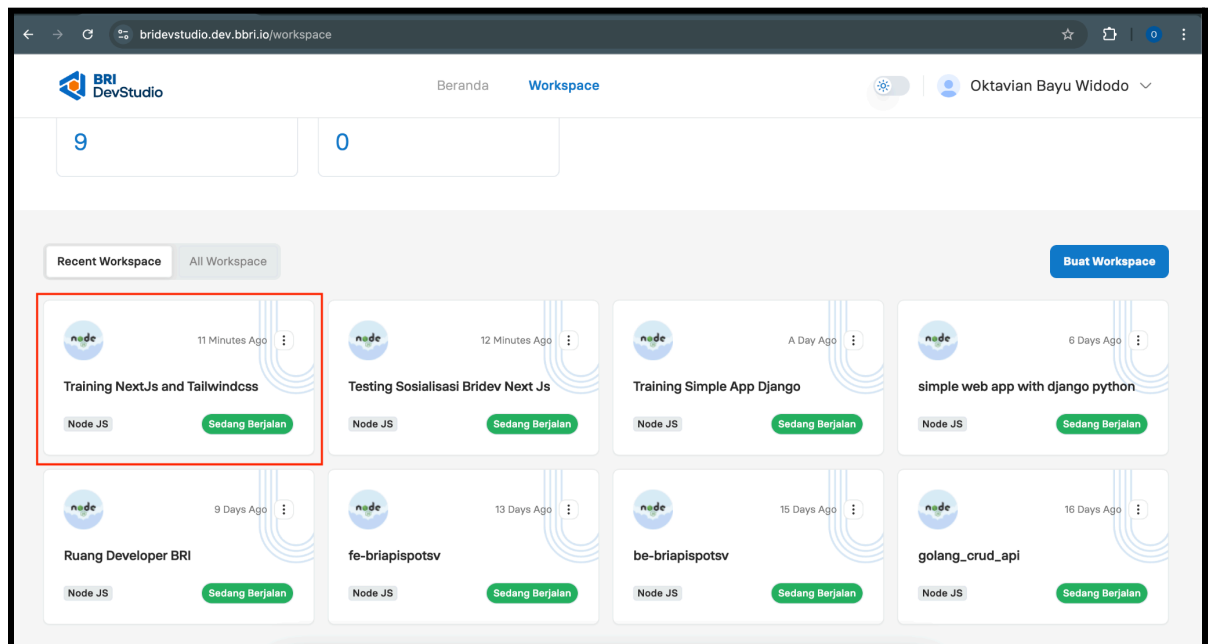
B.2 Inisialisasi Project

Untuk membuat struktur project tersebut, langkahnya adalah sebagai berikut:

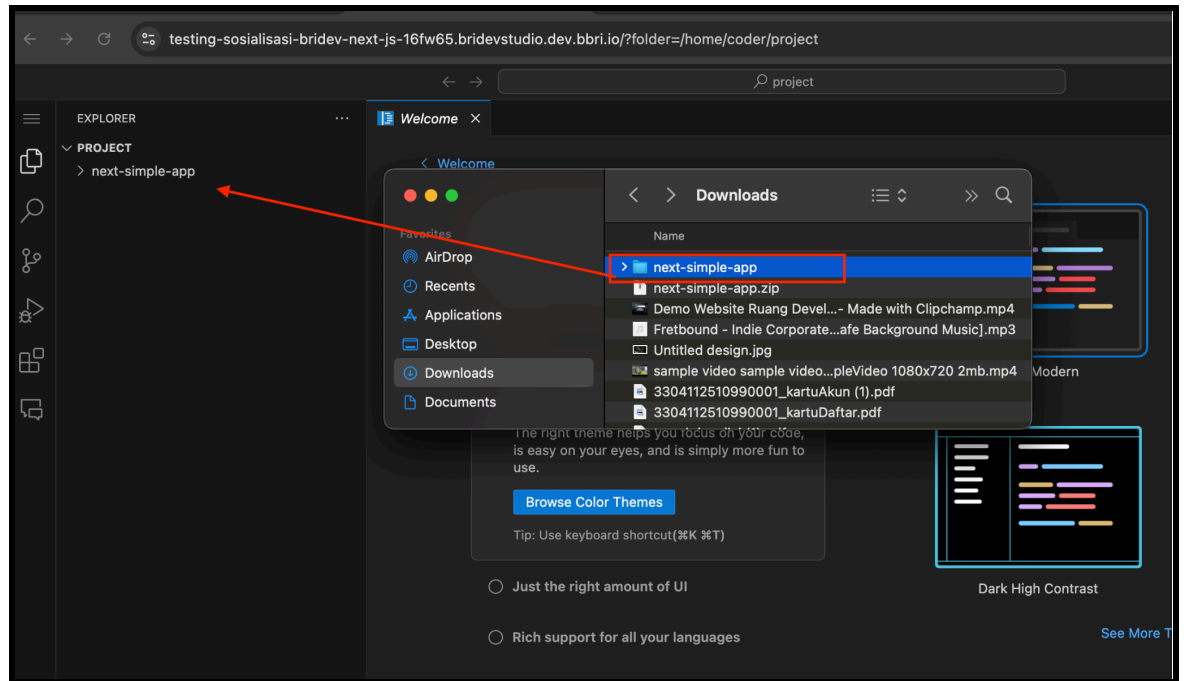
1. Buka url <https://bridevstudio.dev.bbri.io/> dan login menggunakan Email atau akun Google
2. Buat Workspace baru di BRI DevStudio.



3. Setelah Workspace terbuat, buka Workspace



4. Download source code (**Lampiran 1**)
5. Drag and drop source code ke dalam workspace yang sudah dibuat



6. Masuk ke dalam directory project

```
$ cd next-simple-app
```

7. Setelah source code di import, jalankan script berikut ini di terminal untuk instalasi node modules yang dibutuhkan

```
$ npm install
```

8. Buka file **next.config.mjs** dan ganti assetPrefix proxy. Untuk mendapatkan full proxy url dapat mengikuti langkah berikut :
 - a. Jalankan **npm run dev** di terminal
 - b. Kemudian klik “Open In Browser”

Note :

- a. Di DATABASE_URL sesuaikan username, password dan database name yang dibuat di langkah 9
- b. Di bagian **NEXT_PUBLIC_URL** masukkan url sama persis dengan url yang ada di assetPrefix tanpa “/” setelah port. Contoh :

```
NEXT_PUBLIC_URL='https://testing-sosialisasi-bridev-next-js-16fw65.bridevstudio.dev.bbri.io/proxy/3000'
```

11. Langkah selanjutnya, lakukan migrate schema dengan perintah berikut

```
$ npx prisma migrate dev --name init
```

12. Langkah terakhir, jalankan local development server menggunakan perintah berikut

```
$ npm run dev
```

13. Apabila ingin melakukan pengujian terhadap API Endpoint, dapat melihat

Lampiran 3

Lampiran 1

Link project :

https://drive.google.com/file/d/1nLVFWWOnDXrkzIGhjKTHfRR_uo2gX-j/view?usp=sharing

Lampiran 2

1. Instalasi DBMS PostgreSQL

a. Update daftar paket

```
$ sudo apt update
```

b. Instal PostgreSQL

```
$ sudo apt install postgresql postgresql-contrib
```

```
o testing-sosialisasi-bridev-next-js-16fw65-code-server-67cbh8hb4% sudo apt install postgresql postgresql-contrib
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  bsd-mailx cron cron-daemon-common exim4-base exim4-config exim4-daemon-light libcommon-sense-perl libevent-2.1-
  libjson-perl libjson-xs-perl libllvm14 liblockfile-bin liblockfile1 libpopt0 libpq5 libsensors-config libsensors5
  libunbound8 libxslt1.1 libz3-4 logrotate postgresql-15 postgresql-client-15 postgresql-client-common postgresql
Suggested packages:
  anacron checksecurity exim4-doc-html | exim4-doc-info eximon4 file spf-tools-perl swaks dns-root-data lm-sensor
  postgresql-doc-15 isag
The following NEW packages will be installed:
  bsd-mailx cron cron-daemon-common exim4-base exim4-config exim4-daemon-light libcommon-sense-perl libevent-2.1-
  libjson-perl libjson-xs-perl libllvm14 liblockfile-bin liblockfile1 libpopt0 libpq5 libsensors-config libsensors5
  libunbound8 libxslt1.1 libz3-4 logrotate postgresql postgresql-15 postgresql-client-15 postgresql-client-common
  postgresql-contrib sysstat
0 upgraded, 31 newly installed, 0 to remove and 48 not upgraded.
Need to get 52.6 MB of archives.
After this operation, 208 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://deb.debian.org/debian bookworm/main amd64 cron-daemon-common all 3.0pl1-162 [12.7 kB]
Get:2 http://deb.debian.org/debian bookworm/main amd64 cron amd64 3.0pl1-162 [73.1 kB]
Get:3 http://deb.debian.org/debian bookworm/main amd64 libjson-perl all 4.10000-1 [87.5 kB]
Get:4 http://deb.debian.org/debian bookworm/main amd64 postgresql-client-common all 248 [35.1 kB]
Get:5 http://deb.debian.org/debian bookworm/main amd64 postgresql-common all 248 [179 kB]
Get:6 http://deb.debian.org/debian bookworm/main amd64 libpopt0 amd64 1.19+dfsg-1 [43.3 kB]
Get:7 http://deb.debian.org/debian bookworm/main amd64 logrotate amd64 3.21.0-1 [62.1 kB]
Get:8 http://deb.debian.org/debian bookworm/main amd64 liblockfile-bin amd64 1.17-1+b1 [20.8 kB]
Get:9 http://deb.debian.org/debian bookworm/main amd64 exim4-config all 4.96-15+deb12u5 [256 kB]
Get:10 http://deb.debian.org/debian bookworm/main amd64 exim4-base amd64 4.96-15+deb12u5 [1,117 kB]
Get:11 http://deb.debian.org/debian bookworm/main amd64 libevent-2.1-7 amd64 2.1.12-stable-8 [180 kB]
Get:12 http://deb.debian.org/debian bookworm/main amd64 libunbound8 amd64 1.17.1-2+deb12u2 [550 kB]
Get:13 http://deb.debian.org/debian bookworm/main amd64 libgnutls-dane0 amd64 3.7.9-2+deb12u3 [406 kB]
Get:14 http://deb.debian.org/debian bookworm/main amd64 libidn12 amd64 1.41-1 [83.8 kB]
Get:15 http://deb.debian.org/debian bookworm/main amd64 exim4-daemon-light amd64 4.96-15+deb12u5 [605 kB]
Get:16 http://deb.debian.org/debian bookworm/main amd64 liblockfile1 amd64 1.17-1+b1 [17.0 kB]
```

Tunggu sampai proses instalasi selesai

c. Cek status layanan PostgreSQL

```
$ sudo service postgresql status
```

- d. Jika Postgresql belum berjalan, dapat menjalankannya menggunakan script berikut :

```
$ sudo service postgresql start
```

Pastikan postgresql berjalan seperti gambar berikut :

```
● testing-sosialisasi-bridev-next-js-16fw65-code-server-67cbh8hb4% sudo service postgresql start
Starting PostgreSQL 15 database server: main.
```

2. Ubah password user postgres (optional)

- a. Masuk ke dalam shell Postgresql

```
$ sudo -u postgres psql
```

- b. Ubah password pengguna postgres

```
\password postgres
```

Berikut adalah contoh hasil untuk ubah password user postgres :

```
testing-sosialisasi-bridev-next-js-16fw65-code-server-67cbh8hb4% sudo -u postgres psql
psql (15.8 (Debian 15.8-0+deb12u1))
Type "help" for help.

postgres=# \password postgres
Enter new password for user "postgres":
Enter it again:
```

- c. Keluar dari shell

```
\q
```


3. Membuat database

- a. Masuk ke dalam shell Postgresql

```
$ sudo -u postgres psql
```

- b. Buat database baru: Ganti `your_database_name` dengan nama database yang diinginkan

```
CREATE DATABASE your_database_name;
```

Berikut adalah contoh hasil pembuatan database :

```
testing-sosialisasi-bridev-next-js-16fw65-code-server-67cbh8hb4% sudo -u postgres psql
psql (15.8 (Debian 15.8-0+deb12u1))
Type "help" for help.

postgres=# CREATE DATABASE testing_next_simple_app;
CREATE DATABASE
```

- c. Cek daftar database: Untuk melihat database yang telah dibuat

```
\l
```

Berikut adalah contoh database yang sudah dibuat :

```
postgres=# \l
```

Name	Owner	Encoding	Collate	Ctype	ICU Locale	Locale Provider	Access privileges
postgres	postgres	UTF8	C.UTF-8	C.UTF-8		libc	
template0	postgres	UTF8	C.UTF-8	C.UTF-8		libc	=c/postgres + postgres=CtC/postgres
template1	postgres	UTF8	C.UTF-8	C.UTF-8		libc	=c/postgres + postgres=CtC/postgres
testing_next_simple_app	postgres	UTF8	C.UTF-8	C.UTF-8		libc	

(4 rows)

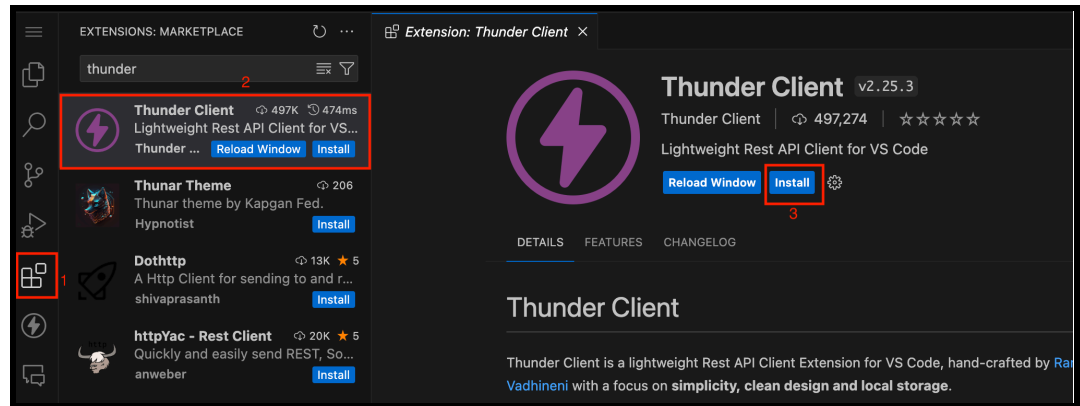
- d. Keluar dari shell

```
\q
```

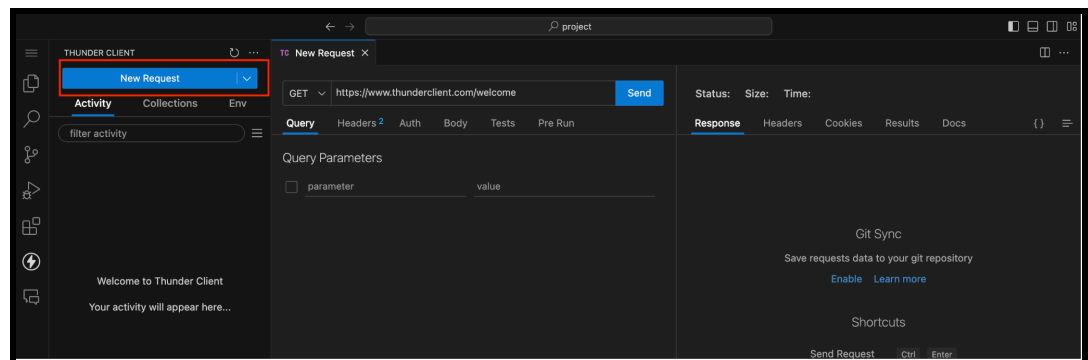
Lampiran 3

Langkah pengujian API Endpoint di BriDev Studio

1. Install Thunder Client Extension di BriDev Studio



2. Untuk menambahkan request, klik “New Request”



3. Lakukan pengujian terhadap API endpoint sesuai dokumentasi berikut

a. Login

URL: `http://localhost:3000/api/auth/login`

Method: POST

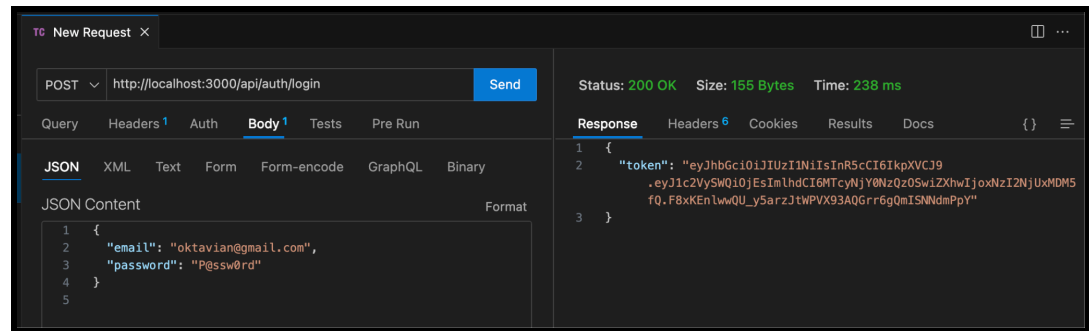
Headers:

- Content-Type: application/json

Body (JSON):

```
{
  "email": "test@example.com",
  "password": "your_password"
}
```

Contoh :



b. Register

URL: http://localhost:3000/api/auth/register

Method: POST

Headers:

- Content-Type: application/json

Body (JSON):

```
{
  "email": "test@example.com",
  "password": "your_password",
  "nik": "123456789"
}
```

c. Users-list

URL: http://localhost:3000/api/users

Method: GET

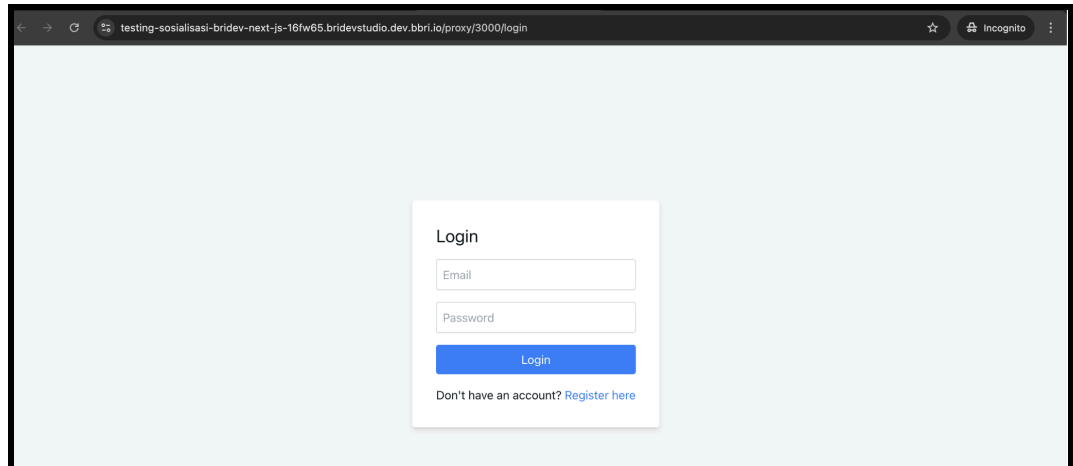
Headers:

- **Authorization:** Bearer `your_token` (ganti `your_token` dengan token yang terima dari login)
- **Content-Type:** application/json (opsional)

Lampiran 4

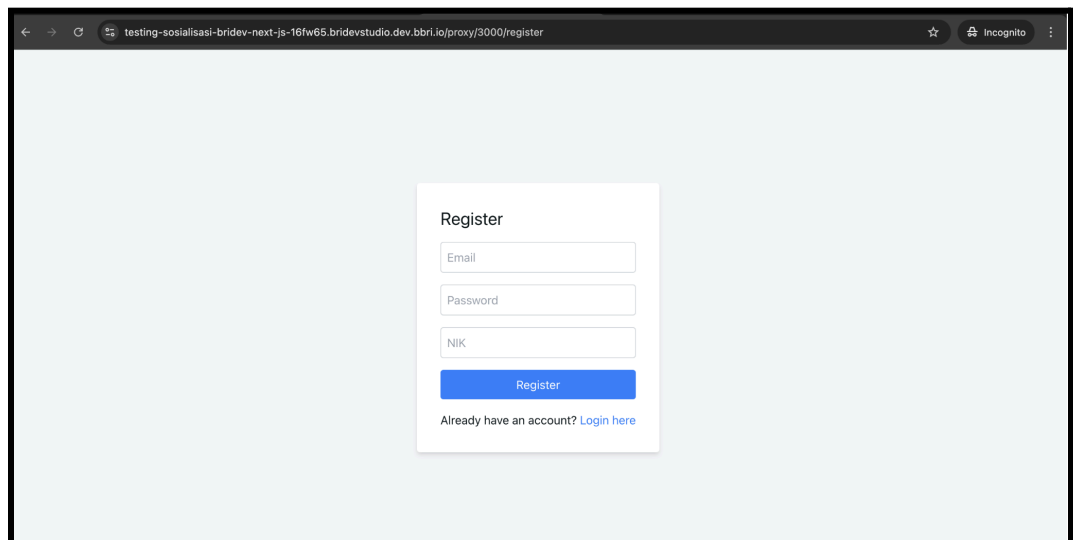
Tampilan Aplikasi

1. Login



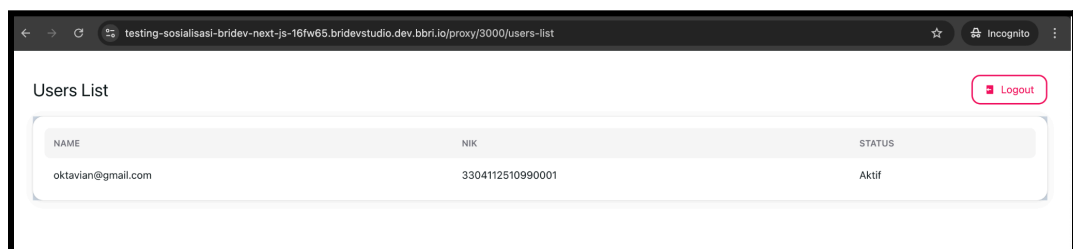
A screenshot of a web browser showing a login page. The browser's address bar displays the URL: `testing-sosialisasi-bridev-next-js-16fw65.bridevstudio.dev.bbri.io/proxy/3000/login`. The page features a light blue background with a white login form in the center. The form has the title "Login" and contains two input fields: "Email" and "Password". Below these fields is a blue "Login" button. At the bottom of the form, there is a link that says "Don't have an account? [Register here](#)".

2. Register



A screenshot of a web browser showing a register page. The browser's address bar displays the URL: `testing-sosialisasi-bridev-next-js-16fw65.bridevstudio.dev.bbri.io/proxy/3000/register`. The page features a light blue background with a white register form in the center. The form has the title "Register" and contains three input fields: "Email", "Password", and "NIK". Below these fields is a blue "Register" button. At the bottom of the form, there is a link that says "Already have an account? [Login here](#)".

3. View all users



A screenshot of a web browser showing a page titled "Users List". The browser's address bar displays the URL: `testing-sosialisasi-bridev-next-js-16fw65.bridevstudio.dev.bbri.io/proxy/3000/users-list`. In the top right corner, there is a red "Logout" button. Below the title, there is a table with three columns: "NAME", "NIK", and "STATUS". The table contains one row of data.

NAME	NIK	STATUS
oktavian@gmail.com	3304112510990001	Aktif